

DAY 3 · LECTURE 1 · 90 MINUTES

The Master Blueprint

Helm: Templating the Whole Fleet

You hand-wrote the manifests. Now watch Hazel stamp out the fleet.



Where we are

- ♦ Days 1–2: you hand-wrote Pods, Deployments, Services, and ConfigMaps — one resource at a time.
- ♦ This morning: **Hazel** (Helm) turns that hand-work into a mass-production mould.
- ♦ Four parts:
 - **Part I** — The tyranny of raw YAML
 - **Part II** — Meet Hazel (Helm basics)
 - **Part III** — One blueprint, many ships (templating)
 - **Part IV** — Launching and upgrading the fleet
- ♦ Then we sail into **Lab 01 — Drafting the Blueprint**.

What you did on Day 2

- ♦ For **one** tier of the 3-tier stack, you wrote:
 - one `Deployment` manifest
 - one `Service` manifest
 - one `ConfigMap` manifest
- ♦ Three resources. One tier. One student. Fully hand-typed.

That was educational. It was also the last time you should ever do it that way.

Now scale that up

```
1 student x 3 tiers x ~2 manifests/tier = 6 files  
30 students x 3 tiers x ~2 manifests/tier = 180 files
```

- ♦ Yesterday you wrote **6** files.
- ♦ Repeat that for the rest of the class: **~180 hand-edited YAML files**.
- ♦ Same tier, same logic — just a different student name, namespace, and image tag on each one.

What goes wrong

- ♦ **Copy-paste drift** — file 12 has the wrong namespace; nobody notices until runtime.
- ♦ **Silent typos** — indentation is wrong; the manifest applies, the Pod never comes up.
- ♦ **No single source of truth** — which version of this file is canonical? They are all different now.
- ♦ **No upgrade path** — changing the image tag means touching all 180 files, one by one.

Raw YAML does not scale. It compounds mistakes.

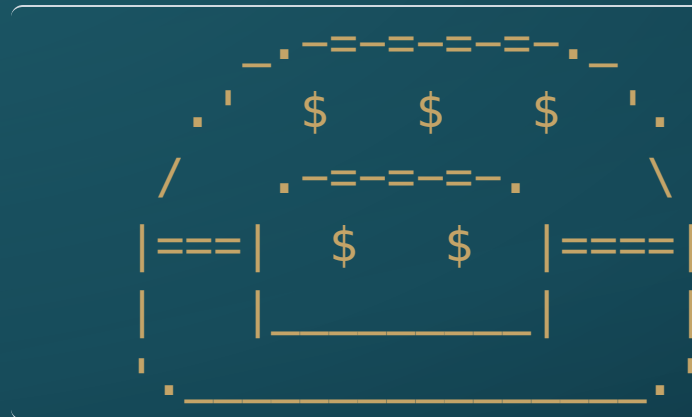
This was never a typing problem

- ♦ You could be more careful with the YAML. It wouldn't help — the problem is structural, not human error.
- ♦ One definition, copied thirty times, is thirty things to keep in sync — forever.
- ♦ The fix is to stop copying and start **templating**: write the stack once, generate the rest.

*There is a better way. Her name is **Hazel**.*

PART II

Meet Hazel



"Hazel don't build ships one plank at a time. She drafts once, stamps a thousand." — the Boatswain

Helm: the package manager for Kubernetes

- ♦ **Helm** is two things at once:
 - a **package manager** — install, upgrade, and remove a whole application with one command
 - a **templating engine** — turn parameterised blueprints into finished Kubernetes YAML
- ♦ The seminar character is **Hazel** — the island's shipwright.
- ♦ Hazel does not hand-build ships. She drafts a **blueprint** once, then stamps out a fleet.

A Chart is a package

- ♦ The unit of work in Helm is a **Chart** — a directory with a specific layout.
- ♦ A chart packages **everything** an application needs: templates, defaults, metadata.
- ♦ You install a chart into a cluster → Helm renders the templates → the resources appear.
- ♦ One chart can deploy to **any** namespace, any cluster — just change the values.

```
helm create island-stack      <-- generate a fresh chart skeleton
helm install  my-stack ./island-stack  <-- stamp it out
helm upgrade  my-stack ./island-stack  <-- update it
helm uninstall my-stack           <-- remove every object it
made
```

The three parts of a Chart

```
island-stack/  
|-- Chart.yaml      the blueprint's name + version  
|-- values.yaml     the knobs you can turn  
+-- templates/      the molds -- YAML with placeholders
```

- ♦ **Chart.yaml** — metadata: name, version, a one-line description.
- ♦ **values.yaml** — all the adjustable settings, in one place, with sensible defaults.
- ♦ **templates/** — your Kubernetes YAML, but with blanks left in.

Hazel reads the knobs, fills in the blanks, and hands finished manifests to the cluster.

Chart.yaml — the name plate

```
apiVersion: v2
name: island-stack
description: The 3-tier island fleet
version: 0.1.0
```

- ♦ **name** — what the chart is called (shows up in `helm list`).
- ♦ **version** — the chart version, not the app version. Bump it when the chart changes.
- ♦ That is all you need to get started. The rest are optional fields.

values.yaml — the blueprint's dials

```
studentName: blackbeard

cache:
  replicaCount: 1
  image: redis:alpine

backend:
  replicaCount: 1
  image: stefanprodan/podinfo:latest

frontend:
  replicaCount: 1
  image: paulbouwer/hello-kubernetes:1.10
```

- ♦ Every knob the template might need is **declared here**, with a default.
- ♦ Change this file → change the whole fleet. No hunting through YAML files.
- ♦ This is the **single source of truth** raw YAML never had.

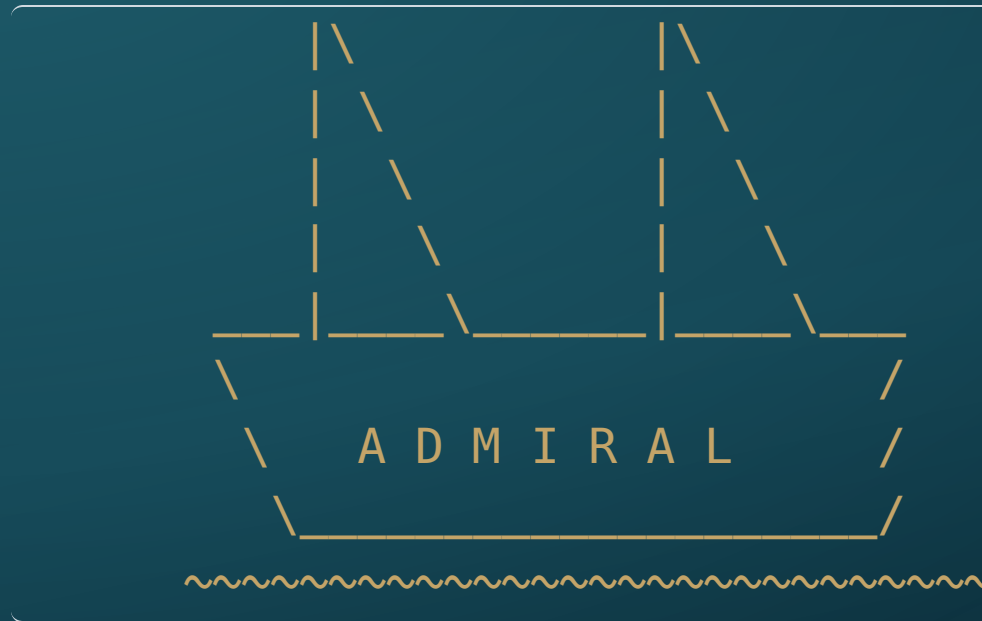
templates/ — the mould

- ♦ Each file in `templates/` is a **Kubernetes manifest with blanks left in**.
- ♦ The blanks are Go-template placeholders: `{{ .Values.something }}`.
- ♦ Helm reads `values.yaml`, fills every blank, and hands the cluster finished YAML.
- ♦ The mould never changes. Only the values do.

Build the mould once. Pour a different set of values in each time.

PART III

One Blueprint, Many Ships



"Same hull, same mold — different name on the bow each time she launches." — the Boatswain

Go-template placeholders

- ◆ Inside any file in `templates/`, a placeholder looks like this:

```
{{ .Values.studentName }}  
{{ .Values.cache.replicaCount }}  
{{ .Values.cache.image }}
```

- ◆ The dot (`.`) is the current context — think of it as "look inside values.yaml here."
- ◆ Helm replaces every `{{ ... }}` with the matching value before the YAML ever reaches the cluster.

Before: hardcoded YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: blackbeard-cache
spec:
  replicas: 1
  selector:
    matchLabels:
      app: blackbeard-cache
  template:
    metadata:
      labels:
        app: blackbeard-cache
    spec:
      containers:
      - name: redis
        image: redis:alpine
```

Hardcoded. Only deploys for one student. 29 copies needed.

After: the same YAML, templated

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Values.studentName }}-cache
spec:
  replicas: {{ .Values.cache.replicaCount }}
  selector:
    matchLabels:
      app: {{ .Values.studentName }}-cache
  template:
    metadata:
      labels:
        app: {{ .Values.studentName }}-cache
    spec:
      containers:
      - name: redis
        image: "{{ .Values.cache.image }}"
```

Change `values.yaml`. Run `helm install`. Done.

What changed — a side by side

	Raw YAML	Helm template
Student name	hardcoded string	<code>{{ .Values.studentName }}</code>
Replica count	hardcoded integer	<code>{{ .Values.cache.replicaCount }}</code>
Image	hardcoded tag	<code>{{ .Values.cache.image }}</code>
To redeploy	edit and re-copy	change <code>values.yaml</code>

The template is the mould. `values.yaml` is the pour.

Parameterise what changes

- ♦ Not everything needs a placeholder — only the things that **vary per environment or per student**.
- ♦ Good candidates: student name, namespace, replica counts, image tags, resource limits.
- ♦ Leave the structural YAML alone — `apiVersion`, `kind`, port numbers that never change.

The rule: if you would need to edit it for a second deployment, put it in `values.yaml`.

Rendering without installing

```
helm template ./island-stack
```

- ♦ Helm fills all the placeholders and **prints the finished YAML to your terminal** — without touching the cluster.
- ♦ Use this to **check your braces** before you install.
- ♦ A typo in `{{ .Values.replicaCount }}` shows up as a render error here, not a mysterious cluster failure later.

Render first. Install when it looks right.

PART IV

Launching & Upgrading the Fleet



*"One order launches a fleet. One order brings it home. That's why we have a harbour-master." —
the Boatswain*

helm install — the first launch

```
helm install my-stack ./island-stack -n blackbeard
```

- ♦ `my-stack` — the **release name**. Helm tracks everything it creates under this name.
- ♦ `./island-stack` — the chart directory.
- ♦ `-n blackbeard` — the namespace to install into.

After this command:

- ♦ Helm renders all the templates.
- ♦ Every Deployment, Service, and ConfigMap appears in the cluster.
- ♦ Helm records the release so it can manage it later.

What a release means

```
helm list -n blackbeard
```

NAME	NAMESPACE	REVISION	STATUS	CHART
my-stack	blackbeard	1	deployed	island-stack-0.1.0

- ♦ Helm **tracks every Kubernetes object** it created for this release.
- ♦ It knows the Deployments, the Services, the ConfigMaps — all of them.
- ♦ This is the key difference from raw `kubectl apply`: **Helm owns the whole stack.**

You do not manage objects one by one. You manage the release.

`helm upgrade` — change the fleet

```
helm upgrade my-stack ./island-stack -n blackbeard
```

- ♦ Edit `values.yaml` (or pass `--set`) → run `helm upgrade` → the whole stack updates.
- ♦ Helm computes the **diff**: only the objects that changed are touched.
- ♦ No re-applying every Service by hand. No wondering which files to run.

```
Before upgrade: cache.replicaCount: 1
```

```
After upgrade:  cache.replicaCount: 3
```

```
Helm patches the Deployment. Nothing else changes.
```

Two ways to change a value

Method	Command	When to use
Edit <code>values.yaml</code>	<code>helm upgrade my-stack ./island-stack</code>	durable change, commit it
<code>--set</code> flag	<code>helm upgrade ... --set cache.replicaCount=5</code>	quick one-off, testing

- ♦ `--set` is the quick test. `values.yaml` is the lasting record.
- ♦ `--set` changes are **not saved** — the next `helm upgrade` from the file will overwrite them.
- ♦ For anything you want to keep, it belongs in the file.

helm rollback — reverse course

```
helm rollback my-stack 1 -n blackbeard
```

- ♦ Helm stores every revision of a release — each `upgrade` increments the number.
- ♦ `rollback` rewinds the cluster to a previous revision in one command.
- ♦ No manually re-applying old files. No digging through `git log`.

REVISION	STATUS	DEPLOYED
1	superseded	initial install
2	superseded	replica bump
3	deployed	current

Bad upgrade? One command, one minute, back to revision 2.

`helm uninstall` — scrap the fleet

```
helm uninstall my-stack -n blackbeard
```

- ♦ Helm removes **every object it created** for the release.
- ♦ No orphaned Services, no stray ConfigMaps drifting in the namespace.
- ♦ Compare to Day 2: you had to `kubectl delete` each file individually and hope you got them all.

One blueprint. One command to build it. One command to remove it.

The full Helm workflow

SCAFFOLD

```
helm create  
island-stack
```

-->

TEMPLATIZE

```
edit templates/  
add {{ .Values }}
```

-->

RENDER

```
helm template ./  
(check your braces)
```

INSTALL

```
helm install  
my-stack ./
```

-->

UPGRADE

```
helm upgrade  
my-stack ./
```

-->

ROLLBACK

```
helm rollback  
my-stack 1
```

Generate, templatize, render, install, upgrade, rollback. That is the whole loop.

Helm tracks history so you don't have to

- ♦ Every install and upgrade is a **numbered revision**.
- ♦ `helm history my-stack -n blackbeard` shows every change, when it happened, and its status.
- ♦ This is the audit trail raw YAML never gave you.

Revision	Status	Chart version	Notes
1	superseded	0.1.0	initial install
2	superseded	0.1.0	replica bump
3	deployed	0.1.1	image tag update

Instructor Superpower

THE MASTER BLUEPRINT

- ♦ **One chart + one `values.yaml` per student** = 30 identical, isolated lab environments.
- ♦ Or: one chart + `--set studentName=...` in a loop = the whole class provisioned from one terminal.
- ♦ A student's environment is broken? `helm uninstall`, `helm install`. Clean state in ten seconds.
- ♦ You don't ask IT to provision thirty environments and wait. You define the stack once and stand up a lesson, a project, or a whole course yourself.

Up next: Lab 01 — Drafting the Blueprint

- ♦ Take your Day 2 3-tier app and **templatize it** into a reusable Helm chart.
- ♦ Define `values.yaml`: student name, replica counts, image tags.
- ♦ `helm template` to check your braces, `helm install` to launch the fleet.
- ♦ `helm upgrade` to change a replica count — **without re-applying a single Service**.

Success: your whole 3-tier stack running in your namespace, managed by a single Helm release — and a `values.yaml` you control.

To the shipyard.



One blueprint. Hazel handles the rest.

